C++ - Module 02

Ad-hoc polymorphism, operator overloading and the Orthodox Canonical class form

Ad-hoc polymorphism allows the same function name or operator to behave differently depending on the types of its arguments (achived through function & operator overloading)

Summary: This document contains the exercises of Module 02 from the C++ modules.

Version: 9.1

Operator overloading allows existing C++ operators to be given custom behaviour for user-defined types
a + b = a.operator+(b)
At least one operand must be a user-def. type.

In C++ → initialize fields directly in the ctor's initialization list
[new line + tab]

Some operators, such as ., ::, ?:, cannot be overloaded.
Op. overloading should preserve intuitive behaviour.
New op. cannot be created; precedence or associavity cannot be changed.

Chapter III

New rules

From now on, all your classes must be designed in the **Orthodox Canonical Form**, unless explicitly stated otherwise. They will then implement the four required member functions below:

- Default constructor
- Copy constructor → invoked when a new object is created based on an existing object
- Copy assignment operator overload
- Destructor

Split your class code into two files. The header file (.hpp/.h) contains the class definition, whereas the source file (.cpp) contains the implementation.

operator = only works on already existing object
assigning a value to an object that already exists

0.1 $(+1) \cdot 1{,}100110011\ldots_2 \cdot 2^{-4} \approx 0.1 \ldots\ldots$ sgn…

sign significand / mantissa exponent

Chapter V

$(-1)^S \times (1.F)_2 \times 2^{E - bias}$

Exercise 00: My First Class in Orthodox Canonical Form

Exercise: 00
My First Class in Orthodox Canonical Form
Directory: *ex00/*
Files to Submit: `Makefile`, `main.cpp`, `Fixed.{h, hpp}`, `Fixed.cpp`
Forbidden: None

You think you know integers and floating-point numbers. How cute.

Please read this 3 pages article (1, 2, 3) to discover that you don't. Go on, read it.

Until today, every number you used in your code was basically either an integer or a floating-point number, or any of their variants (`short`, `char`, `long`, `double`, and so forth). After reading the article above, it's safe to assume that integers and floating-point numbers have opposite characteristics.

IEEE 754 Standard

But today, things will change. You are going to discover a new and awesome number type: **fixed-point numbers**! Forever missing from the scalar types of most languages, fixed-point numbers offer a valuable balance between performance, accuracy, range and precision. That explains why fixed-point numbers are particularly applicable to computer graphics, sound processing or scientific programming, just to name a few.

As C++ lacks fixed-point numbers, you're going to add them. This article from Berkeley is a good start. If you have no idea what Berkeley University is, read this section of its Wikipedia page.

if (this!=&other) -> self-assignment guard; needed when the operator frees or modifies the objects own resources before it has finished reading from other

Create a class in Orthodox Canonical Form that represents a fixed-point number:

- Private members:
  - An **integer** to store the fixed-point number value. _value
  - A **static constant integer** to store the number of fractional bits. Its value will always be the integer literal 8. _fractionalBits -> they call it "literal" because it is a value literally in the code, rather than, for example variable or even MACRO
- Public members:
  - A default constructor that initializes the fixed-point number value to 0.
  - A copy constructor.
  - A copy assignment operator overload.
  - A destructor.
  - A member function `int getRawBits( void ) const;` that returns the raw value of the fixed-point value.
  - A member function `void setRawBits( int const raw );` that sets the raw value of the fixed-point number.

value stored internally, w/o converting or interpreting it.

Running this code:

```
#include <iostream>

int         main( void ) {

  Fixed a;
  Fixed b( a );  -> copy ctor
  Fixed c;

  c = b;  -> copy assignment operator

  std::cout << a.getRawBits() << std::endl;
  std::cout << b.getRawBits() << std::endl;
  std::cout << c.getRawBits() << std::endl;

  return 0;
}
```

Should output something similar to:

```
$> ./a.out
Default constructor called
Copy constructor called
Copy assignment operator called // <-- This line may be missing depending on your implementation
getRawBits member function called
Default constructor called
Copy assignment operator called
getRawBits member function called
getRawBits member function called
0
getRawBits member function called
0
getRawBits member function called
0
Destructor called
Destructor called
Destructor called
$>
```

$$\text{real value} = \frac{\text{raw value}}{2 \wedge \text{fractional bits}}$$

int = fixed >> 2^fBits   fixed (raw) = int << 2^fBits

Chapter VI

Exercise 01: Towards a more useful fixed-point number class

	Exercise01
Towards a more useful fixed-point number class	
Directory: ex01/	
Files to Submit: `Makefile, main.cpp, Fixed.{h, hpp}, Fixed.cpp`	
Authorized: `roundf (from <cmath>)`	

float roundf (float num):   <0.5 → 0

result is of type float but it is an int value represented as a float (eg 2.0f)   >=0.5 → 1

The previous exercise was a good start, but our class is pretty useless. It can only represent the value `0.0`.

Add the following public constructors and public member functions to your class:

- ? A constructor that takes a **constant integer** as a parameter.
  It converts it to the corresponding fixed-point value. The fractional bits value should be initialized to 8, like in exercise 00.
- A constructor that takes a **constant floating-point number** as a parameter.
  It converts it to the corresponding fixed-point value. The fractional bits value should be initialized to 8, like in exercise 00.
- A member function `float toFloat( void ) const;`
  that converts the fixed-point value to a floating-point value.
- A member function `int toInt( void ) const;` truncate ~~or round~~?
  that converts the fixed-point value to an integer value.

And add the following function to the **Fixed** class files:

- An overload of the insertion («) operator that inserts a floating-point representation (to float) of the fixed-point number into the output stream object passed as a parameter.

Running this code:

```
#include <iostream>

int     main( void ) {

    Fixed       a;
    Fixed const b( 10 );
    Fixed const c( 42.42f );
    Fixed const d( b );

    a = Fixed( 1234.4321f );

    std::cout << "a is " << a << std::endl;
    std::cout << "b is " << b << std::endl;
    std::cout << "c is " << c << std::endl;
    std::cout << "d is " << d << std::endl;

    std::cout << "a is " << a.toInt() << " as integer" << std::endl;
    std::cout << "b is " << b.toInt() << " as integer" << std::endl;
    std::cout << "c is " << c.toInt() << " as integer" << std::endl;
    std::cout << "d is " << d.toInt() << " as integer" << std::endl;

    return 0;
}
```

Should output something similar to:

```
$> ./a.out
Default constructor called
Int constructor called
Float constructor called
Copy constructor called
Copy assignment operator called
Float constructor called
Copy assignment operator called
Destructor called
a is 1234.43
b is 10
c is 42.4219
d is 10
a is 1234 as integer
b is 10 as integer
c is 42 as integer
d is 10 as integer
Destructor called
Destructor called
Destructor called
Destructor called
$>
```

When storing a float in _value → multiply by 256

when converting _value back to float → divide post - returns the previous value of an object, typically by value [copy by]

++c Fixed& ref
c++ Fixed val

lhs -> left hand side rhs -> right hand side

for members lhs is the object on which the operator is called
rhs is passed as an argument to the operator

Chapter VII

Exercise 02: Now we're talking

	Exercise02
Now we're talking	
Directory: ex02/	
Files to Submit: `Makefile, main.cpp, Fixed.{h, hpp}, Fixed.cpp`	
Authorized: `roundf (from <cmath>)`	

Add public member functions to your class to overload the following operators:

- The 6 comparison operators: >, <, >=, <=, ==, and !=. raw
- The 4 arithmetic operators: +, -, *, and /. raw +&- raw
- The 4 increment/decrement (pre-increment and post-increment, pre-decrement and post-decrement) operators, which will increase or decrease the fixed-point value by the smallest representable ϵ, such that $1 + \epsilon > 1$. -> raw + 1 not 256
  take the smallest ~~possible~~ possible step

Add these four public overloaded member functions to your class:

- A static member function `min` that takes two references to fixed-point numbers as parameters, and returns a reference to the smallest one. Fixed&
- A static member function `min` that takes two references to **constant** fixed-point numbers as parameters, and returns a reference to the smallest one. const Fixed&
- A static member function `max` that takes two references to fixed-point numbers as parameters, and returns a reference to the greatest one.
- A static member function `max` that takes two references to **constant** fixed-point numbers as parameters, and returns a reference to the greatest one.

constness -> const corectness

It's up to you to test every feature of your class. However, running the code below:

```
#include <iostream>

int     main( void ) {

    Fixed           a;
    Fixed const     b( Fixed( 5.05f ) * Fixed( 2 ) );

    std::cout << a << std::endl;
    std::cout << ++a << std::endl;
    std::cout << a << std::endl;
    std::cout << a++ << std::endl;
    std::cout << a << std::endl;

    std::cout << b << std::endl;

    std::cout << Fixed::max( a, b ) << std::endl;

    return 0;
}
```

Should output something like (for greater readability, the constructor/destructor messages are removed in the example below):

```
$> ./a.out
0
0.00390625
0.00390625
0.00390625
0.0078125
10.1016
10.1016
$>
```

If you ever do a division by 0, it is acceptable that the program crashes

multiply $\frac{raw_1 \cdot raw_2}{256}$

divide $\frac{raw_1 \cdot 256}{raw_2}$

For comparison op. only two of them need to be implemented as a whole == and < or >

!= -> !(a==b)

a>b -> b<a if a is bigger than a the b is smaller than a

a<=b -> !(a>b) -> (!b<a)

a>=b -> !(a<b)